OOPS CONCEPTS

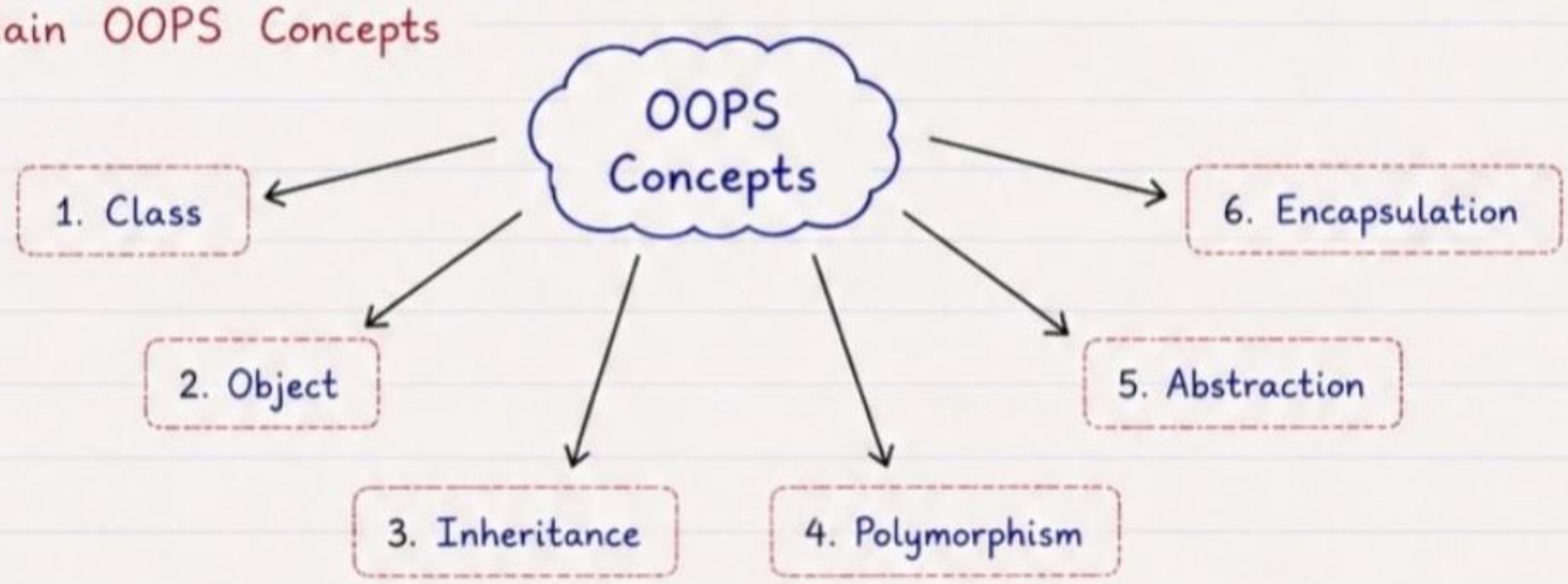
✱ What is OOPS ?

OOPS (Object-Oriented Programming System) is a programming paradigm based on the concept of "Objects", which can contain data in the form of fields (attributes) and code in the form of procedures (methods).

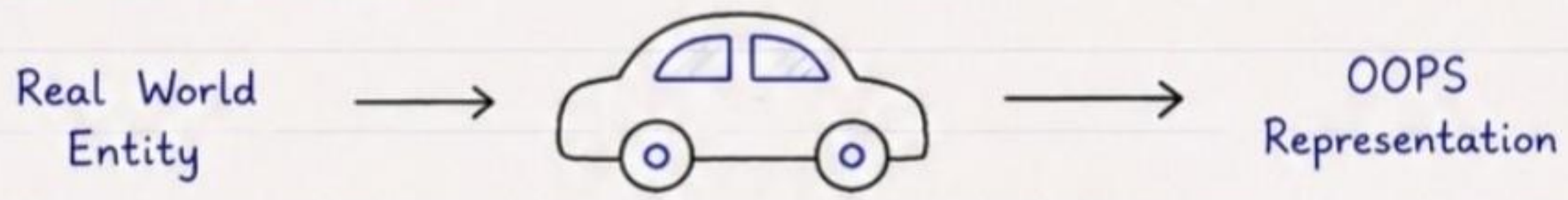
✱ Why OOPS ?

- It makes programs easier to understand, maintain and modify.
- It provides data security by data hiding.
- It supports reusability through inheritance.
- It allows flexibility through polymorphism.
- It helps in modeling real-world entities.

✱ Main OOPS Concepts



✱ OOPS in Real World



Car has properties (color, speed) and behaviors (start, stop)

✱ Summary

OOPS is a powerful programming paradigm that uses objects and classes to design modular, reusable and scalable applications. The above concepts work together to make the code efficient and easy to manage.

Remember:
Think Classes,
Create Objects,
Build Solutions ! ☆

OOPS CONCEPT FULL HANDBOOK

1. CLASS

★ What is a Class ?

A class is a **blueprint** or **template** for creating objects.
It defines a set of **attributes (data)** and **methods (functions)** that the objects of the class will have.

★ Key Points

- A class does **not** occupy memory.
- It is a **user-defined data type**.
- It acts as a **blueprint** for creating objects.
- It helps in **data hiding** and **code reusability**.

★ Syntax (in Java)

```
class ClassName {
        // Attributes
        dataType attribute1;
        dataType attribute2;
        ...
        // Methods
        returnType methodName(parameters) {
            // method body
        }
}
```

→ Class Keyword

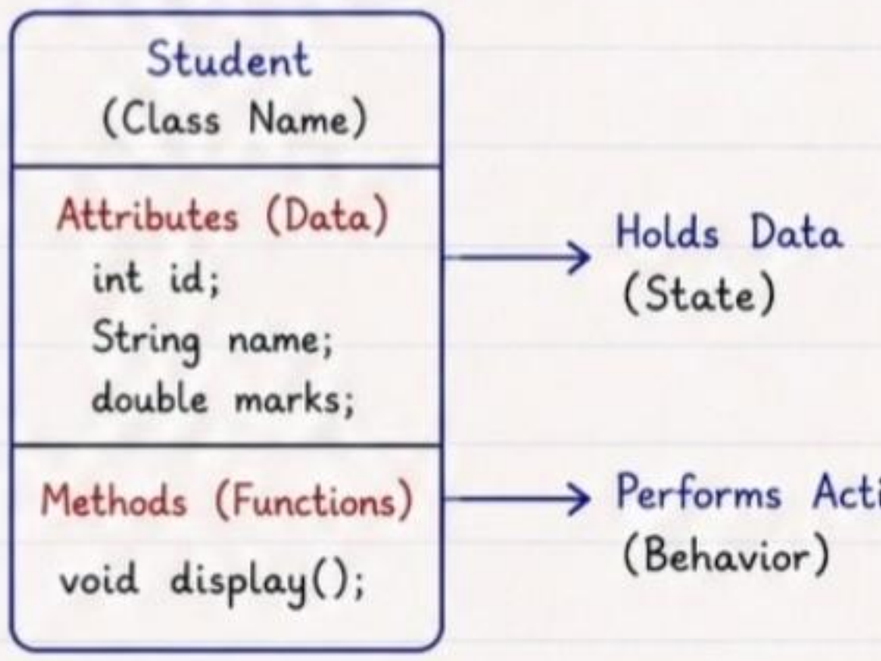
→ Attributes / Variables (Data Members)

→ Methods / Functions (Member Methods)

★ Example

```
class Student {
        // Attributes
        int id;
        String name;
        double marks;
        // Methods
        void display() {
            System.out.println("ID: " + id);
            System.out.println("Name: " + name);
            System.out.println("Marks: " + marks);
        }
}
```

Class Structure



Remember :

A class is just a **template**. No memory is allocated when a class is created. Memory is allocated **only when objects are created** from the class.

Example :

```
Student s1 = new Student();
// Object creation
```

OOPS CONCEPT FULL HANDBOOK

2. OBJECT

★ What is an Object ?

An object is a real-world entity which is an instance of a class.
It has state (data) and behavior (methods).

★ Key Points

- An object is created from a class.
- It occupies memory.
- Each object has its own copy of data.
- Objects can interact by calling methods.
- Multiple objects can be created from the same class.

★ Syntax (in Java)

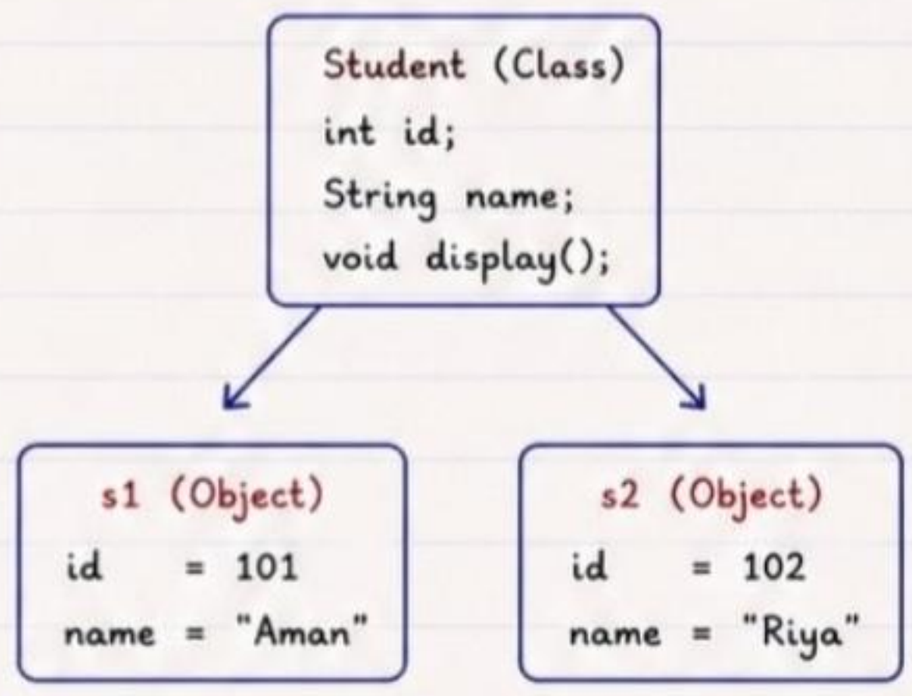
```
ClassName objectName = new ClassName();
```

- Class Name
- Object Name (Reference)
- new Keyword
- Constructor

★ Example

```
class Student {
    int id;
    String name;

    void display() {
        System.out.println(id + " - " + name);
    }
}
public class Main {
    public static void main(String[] args) {
        // Creating objects
        Student s1 = new Student();
        Student s2 = new Student();

        // Setting values
        s1.id = 101;
        s1.name = "Aman";
        s2.id = 102;
        s2.name = "Riya";

        // Calling method
        s1.display();    // Output: 101 - Aman
        s2.display();    // Output: 102 - Riya
    }
}
```

Object Diagram



Here, s1 and s2 are objects (instances) of class Student.
Each object has its own data.

Remember :

- Class is a blueprint.
- Object is a real entity created from the class.
- Many objects can be created from one class.

Example in Real Life :

Class : Mobile
Objects : iphone, samsung, oneplus (each mobile has its own details and behavior)

OOPS CONCEPT FULL HANDBOOK

3. METHOD

★ What is a Method ?

A method is a block of code that performs a specific task.
It is defined inside a class and is used to define the behavior of objects.

★ Key Points

- A method is a function that belongs to a class.
- It is used to perform some operation.
- It can access and modify the data (attributes) of the class.
- Methods improve code modularity and reusability.
- Methods can return a value or may not return anything (void).

★ Syntax (in Java)

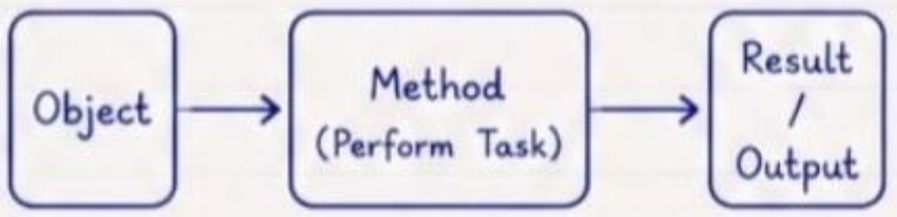
```
returnType methodName(parameters) {
        // method body
        // statements
        return value;   // optional
}
```

- `returnType` → Return type of method (int, void, String, etc.)
- `methodName` → Name of the method
- `parameters` → Parameters passed to the method (optional)
- `{ ... }` → Method body

★ Example

```
class Calculator {
    int add(int a, int b) {
        int sum = a + b;
        return sum;
    }
    void showMessage() {
        System.out.println("Hello, Java!");
    }
}
public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        int result = c.add(10, 20);   // calling method with return
        System.out.println("Sum = " + result);
        c.showMessage();              // calling void method
    }
}
```

How Method Works ?



Object calls the method.
Method performs the task.
Method returns the result (if any).

Remember :

- Method is the behavior of an object.
- It is always defined inside a class.
- A method can return a value or nothing.
- Good method names describe what the method does.

Example in Real Life :

Think of a remote control.
Pressing a button (method) performs an action like changing channel, increasing volume, etc.
Each button has a specific function.

4. CONSTRUCTOR

★ What is a Constructor ?

A constructor is a special method in a class that is used to initialize objects. It is called automatically when an object is created.

★ Key Points

- Constructor name is same as class name.
- It has no return type, not even void.
- It is called automatically when object is created.
- Used to initialize object's data.
- A class can have multiple constructors (constructor overloading).
- If no constructor is written, Java provides a default constructor.

★ Syntax (in Java)

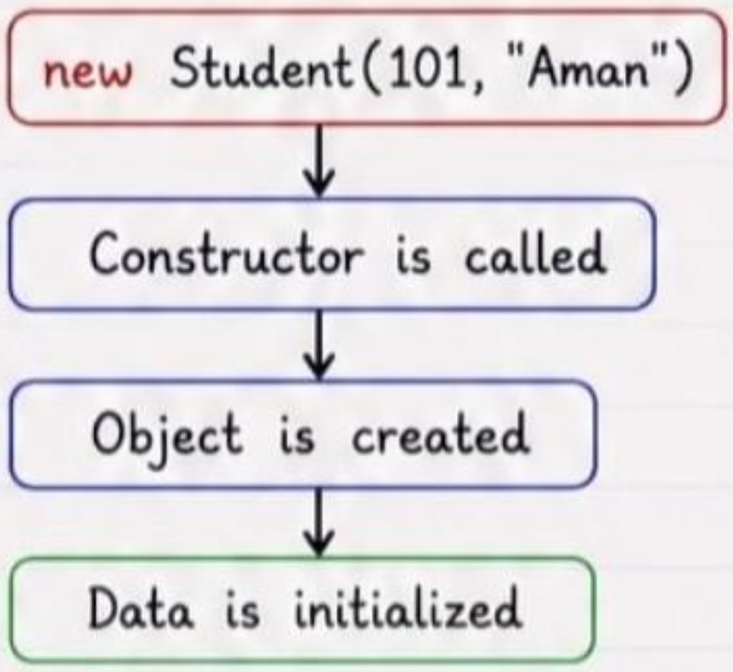
```
class ClassName {
      ClassName(parameters) {
          // initialization code
      }
}
```

→ Constructor name
→ Parameters (optional)
→ Constructor body

★ Example

```
class Student {
      int id;
      String name;
      // Constructor
      Student(int id, String name) {
          this.id = id;
          this.name = name;
      }
      void display() {
          System.out.println(id + " - " + name);
      }
}
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(101, "Aman");
        Student s2 = new Student(102, "Riya");
        s1.display();    // Output: 101 - Aman
        s2.display();    // Output: 102 - Riya
    }
}
```

How Constructor Works ?



Types of Constructors

- Default Constructor (no-arg)
- Parameterized Constructor
- Copy Constructor (via parameter)

Remember :

- Constructor runs only once when object is created.
- It helps to initialize object

Example in Real Life :

When you register in any website, your details (name, email, password) are filled automatically in the form

OOPS CONCEPT FULL HANDBOOK

5. INHERITANCE

★ What is Inheritance ?

Inheritance is a mechanism where a new class (child/subclass) acquires the properties and behaviors (fields and methods) of an existing class (parent/superclass). It promotes code reusability and establishes an IS-A relationship.

★ Key Points

- The class whose members are inherited is called superclass (parent class).
- The class that inherits is called subclass (child class).
- The subclass can have its own members in addition to inherited members.
- Inheritance supports code reusability and extensibility.
- Java does not support multiple inheritance with classes (but supports it using interfaces).

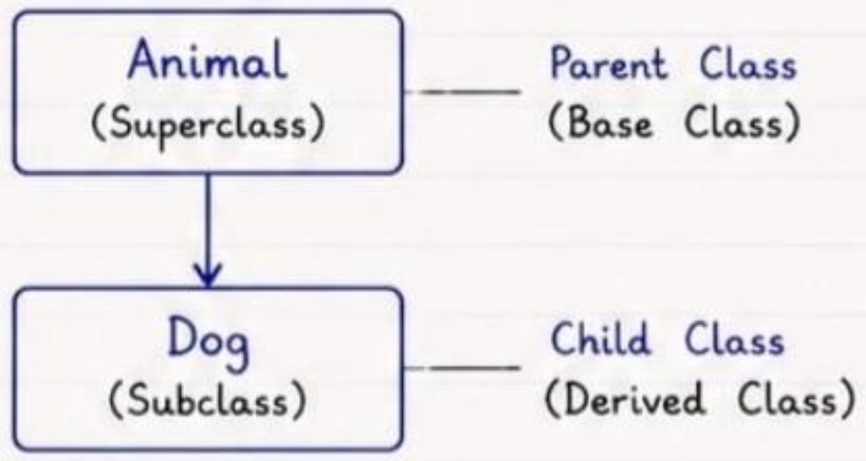
★ Syntax (in Java)

```
class SuperClass {                          ──→ Parent / Superclass
    // fields and methods
}
class SubClass extends SuperClass {         ──→ Child / Subclass
    // own fields and methods
}
```

★ Example (in Java)

```
// Parent Class (Superclass)
class Animal {
    String color = "Brown";
    void eat() {
        System.out.println("Animal is eating");
    }
}
// Child Class (Subclass)
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}
// Main Class
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();        // inherited method
        d.bark();       // own method
        System.out.println(d.color); // inherited field
    }
}
```

Inheritance Diagram



How it Works ?

1. Dog is a Animal. (IS-A Relationship)
2. Dog inherits color and eat() from Animal.
3. Dog can use inherited members and also define its own members.

Benefits	Remember
• Code Reusability	✓ Use extends keyword to inherit.
• Easy Maintenance	✓ Private members are not inherited.
• Extensibility	✓ Constructors are not inherited.
• Better Organization	✓ Inheritance represents IS-A relationship.

6. POLYMORPHISM

★ What is Polymorphism ?

Polymorphism means "many forms". It allows one interface to be used for different types.

In Java, polymorphism is achieved by Method Overloading and Method Overriding.

★ Types of Polymorphism

1. Compile-time Polymorphism (Method Overloading)
2. Run-time Polymorphism (Method Overriding)

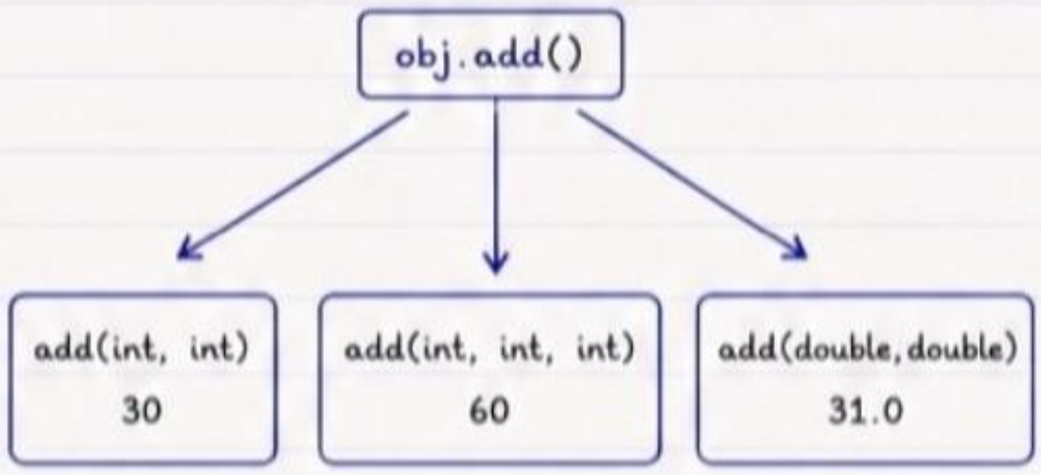
1. Compile-time Polymorphism (Method Overloading)

- Occurs in the same class.
- Same method name but different parameter list (number, type or order).
- It is resolved at compile time.

Example (in Java)

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator obj = new Calculator();
        System.out.println(obj.add(10, 20));      // 30
        System.out.println(obj.add(10, 20, 30));  // 60
        System.out.println(obj.add(10.5, 20.5));  // 31.0
    }
}
```

How it Works ?



The compiler decides which method to call based on the arguments passed.

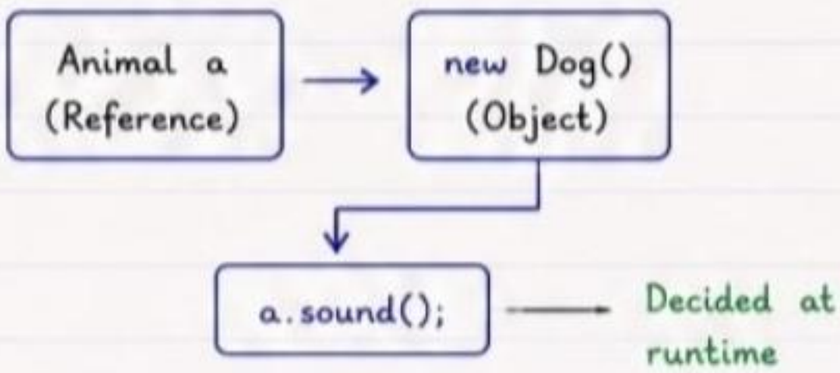
2. Run-time Polymorphism (Method Overriding)

- Occurs in different classes using inheritance.
- Same method name, same parameters.
- It is resolved at runtime.

Example (in Java)

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();   // Reference of parent, object of child
        a.sound();              // Calls overridden method
    }
}
```

How it Works ?



JVM decides at runtime which method to call based on the actual object.

Key Difference

Feature	Method Overloading	Method Overriding
Type	Compile-time	Run-time
Where	Same class	Different class (Inheritance)

Remember

- Overloading increases readability.
- Overriding achieves runtime polymorphism.
- @Override annotation is good practice.

OOPS CONCEPT FULL HANDBOOK

8. ENCAPSULATION

★ What is Encapsulation ?

Encapsulation is the mechanism of wrapping data (variables) and code (methods) together as a single unit and restricting direct access to some of the object's components. It helps in data hiding and protects the data from unauthorized access.

★ Key Points

- We achieve encapsulation by using access modifiers.
- Class variables should be private.
- Access to variables is provided through public getter and setter methods.
- It protects data from being modified by accident.
- It improves code maintainability and flexibility.

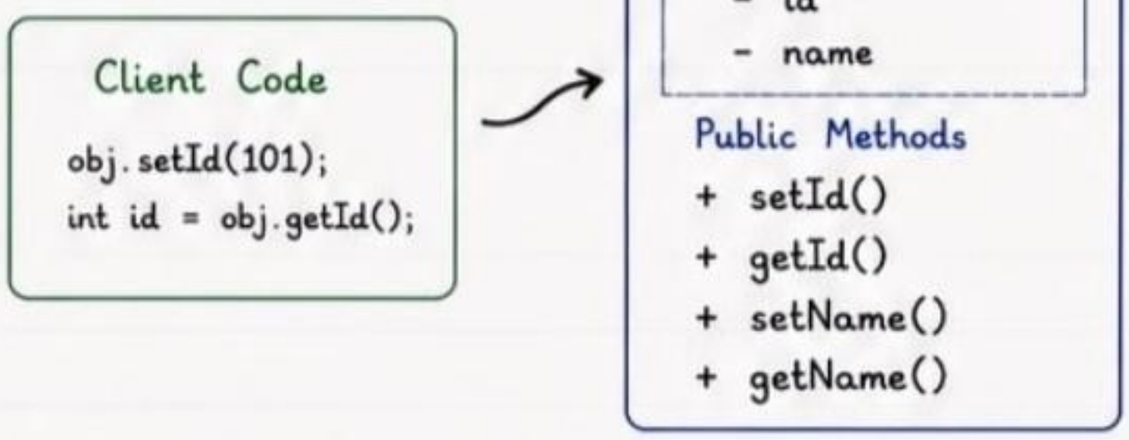
★ Syntax (in Java)

```
class ClassName {
    private dataType variable;    // private variable

    // getter method
    public dataType getVariable() {
        return variable;
    }

    // setter method
    public void setVariable(dataType value) {
        variable = value;
    }
}
```

★ Example (in Java)

```
class Student {
    private int id;
    private String name;

    // Getter for id
    public int getId() {
        return id;
    }

    // Setter for id
    public void setId(int id) {
        this.id = id;
    }

    // Getter for name
    public String getName() {
        return name;
    }

    // Setter for name
    public void setName(String name) {
        this.name = name;
    }
}
```

★ How it Works ?



Benefits

- Protects data from unauthorized access.
- Improves security.
- Allows controlled access to data.
- Makes code easy to maintain and modify.

Remember

- Make variables private.
- Provide public getter and setter methods.
- Access and modify data only through methods.